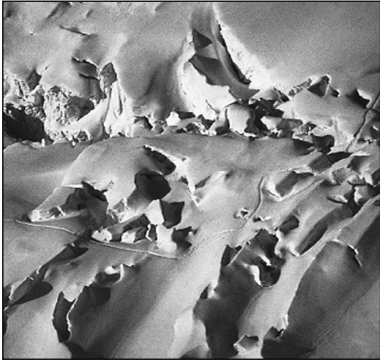


5



Analysis

I am Temp1 with a name, if I could only remember it.
—Anonymous Programmer

Analysis results in a model of the system that aims to be correct, complete, consistent, and verifiable. Developers formalize the system specification produced during requirements elicitation and examine in more detail boundary conditions and exceptional cases. Developers correct and clarify the system specification if any errors or ambiguities are found. The client and the user are usually involved in this activity, especially when the system specification needs to be changed and when additional information needs to be gathered.

In object-oriented analysis, developers build a model describing the application domain. For example, the analysis model of a watch describes how the watch represents time (e.g., Does the watch know about leap years? Does it know about the day of the week? Does it know about the phases of the moon?) The analysis model is then extended to describe how the actors and the system interact to manipulate the application domain model (e.g., How does the watch owner reset the time? How does the watch owner reset the day of the week?). Developers use the analysis model, together with nonfunctional requirements, to prepare for the architecture of the system developed during high-level design (Chapter 6, *System Design*).

In this chapter, we discuss the analysis activities in more detail. We focus on the identification of objects, their behavior, their relationships, their classification, and their organization. We review briefly non-object-oriented analysis presentations and methods. Finally, we describe management issues related to analysis in the context of a multiteam development project.



5.1 Introduction: An Optical Illusion

In 1915, Rubin exhibited a drawing similar to Figure 5-1 to illustrate the concept of multistable images. What do you see? Two faces looking at each other? If you focus more closely on the white area, you can see a vase instead. Once you are able to perceive both shapes individually, it is easier to switch back and forth between the vase and the faces.

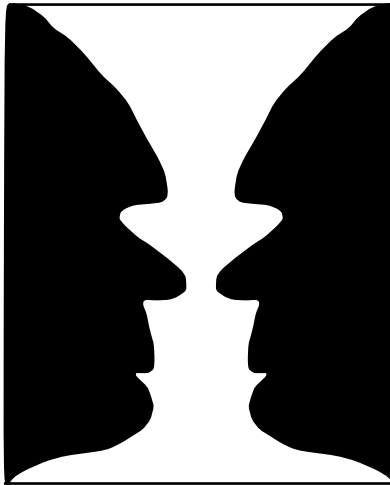


Figure 5-1 Ambiguity: what do you see?

If the drawing in Figure 5-1 had been a system specification, which models should you have constructed? Specifications, like multistable images, contain ambiguities caused by the inaccuracies inherent to natural language and by the assumptions of the specification authors. For example, a quantity specified without a unit is ambiguous (e.g., the “Feet or Miles?” example in Section 4.1), a time without time zone is ambiguous (e.g., scheduling a phone call between different countries).

Formalization helps identify areas of ambiguity as well as inconsistencies and omissions in a system specification. Once developers identify problems with the specification, they address them by eliciting more information from the users and the client. Requirements elicitation and analysis are iterative and incremental activities that occur concurrently.

5.2 An Overview of Analysis

Analysis focuses on producing a model of the system, called the analysis model, which is correct, complete, consistent, and verifiable. Analysis is different from requirements elicitation in that developers focus on structuring and formalizing the requirements elicited from users



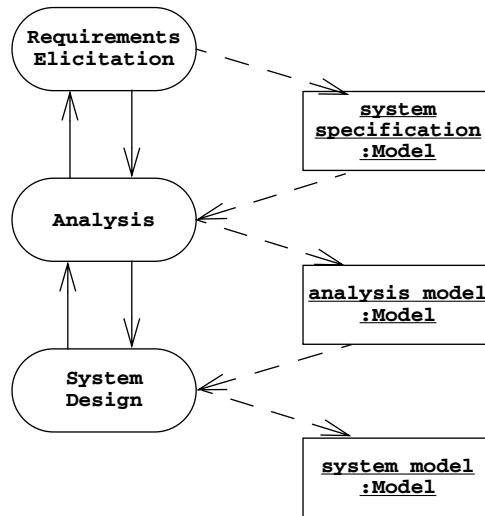


Figure 5-2 Products of requirements elicitation and analysis (UML activity diagram).

(Figure 5-2). Although the analysis model may not be understandable to the users and the client, it helps developers verify the system specification produced during requirements elicitation.

There is a natural tendency for users and developers to postpone difficult decisions until later in the project. A decision may be difficult because of lack of domain knowledge, lack of technological knowledge, or simply because of disagreements among users and developers. Postponing decisions enables the project to move on smoothly and avoid confrontation with reality or peers. Unfortunately, difficult decisions will eventually need to be made, often at higher cost when intrinsic problems are discovered during testing, or worse, during user evaluation. Translating a system specification into a formal or semiformal model forces developers to identify and resolve difficult issues early in the development.

The **analysis model** is composed of three individual models: the **functional model**, represented by use cases and scenarios, the **analysis object model**, represented by class and object diagrams, and the **dynamic model**, represented by statechart and sequence diagrams (Figure 5-3). In the previous chapter, we described how to elicit requirements from the users and describe them as use cases and scenarios. In this chapter, we describe how to refine the functional model and derive the object and the dynamic model. This leads to a more precise and complete specification as details are added to the analysis model. We conclude the chapter by describing management activities related to analysis. In the next section, we first define the main concepts of analysis.

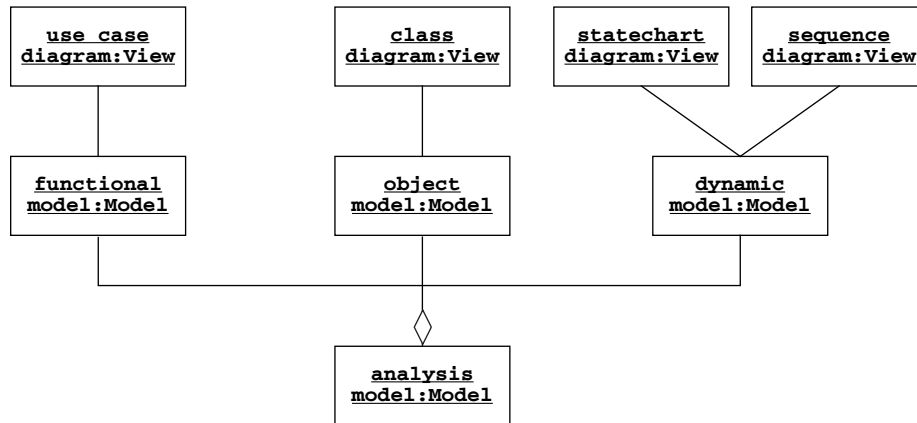


Figure 5-3 The analysis model is composed of the functional model, the object model, and the dynamic model. In UML, the functional model is represented with use case diagrams, the object model with class diagrams, and the dynamic model with statechart and sequence diagrams.

5.3 Analysis Concepts

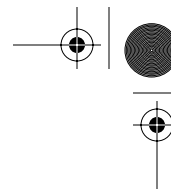
In this section, we describe the main analysis concepts used in this chapter. In particular, we describe:

- entity, boundary, and control objects (Section 5.3.1)
- association multiplicity (Section 5.3.2)
- qualified associations (Section 5.3.3)
- generalization (Section 5.3.4)

5.3.1 Entity, Boundary, and Control Objects

The analysis object model consists of entity, boundary, and control objects [Jacobson et al., 1999]. **Entity objects** represent the persistent information tracked by the system. **Boundary objects** represent the interactions between the actors and the system. **Control objects** represent the tasks that are performed by the user and supported by the system. In the 2Bwatch example, Year, Month, Day are entity objects; ButtonBoundary and LCDDisplayBoundary are boundary objects; ChangeDateControl is a control object that represents the activity of changing the date by pressing combinations of buttons.

Modeling the system with entity, boundary, and control objects has several advantages. First, it provides developers with simple heuristics to distinguish different, but related concepts. For example, the time that is tracked by a watch has different properties than the display that depicts the time. Differentiating between boundary and entity objects forces that distinction: The



time that is tracked by the watch is represented by the `Time` object. The display is represented by the `LCDDisplayBoundary`. Second, the three object type approach results in smaller and more specialized objects. Third, the three object type approach leads to models that are more resilient to change: The interface to the system (represented by the boundary objects) is more likely to change than its basic functionality (represented by the entity and control objects).

To distinguish between different types of objects, UML provides the stereotype mechanism to enable the developer to attach such meta information to modeling elements. For example, in Figure 5-4, we attach the `<<control>>` stereotype to the `ChangeDateControl` object. In addition to stereotypes, we may also use naming conventions for clarity and recommend distinguishing the three different types of objects on a syntactical basis: Boundary objects may have the suffix `Boundary` appended to their name; control objects may have the suffix `Control` appended to their name; entity objects usually do not have any suffix appended to their name. Another benefit of this naming convention is that the type of the class is represented even when the UML stereotype is not available, for example, when examining only the source code.

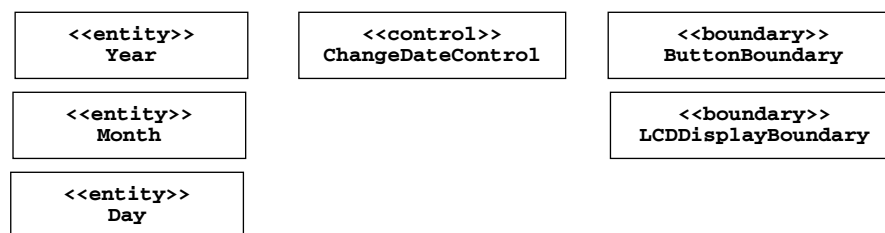


Figure 5-4 Analysis classes for the 2Bwatch example.

5.3.2 Association Multiplicity Revisited

As we saw in Chapter 2, *Modeling with UML*, the end of an association can be labeled by a set of integers called **multiplicity**. The multiplicity indicates the number of links that can legitimately originate from an instance of the class connected to the association end. For example, in Figure 5-5, a `2Bwatch` has exactly two `Buttons` and one `LCDDisplay`.

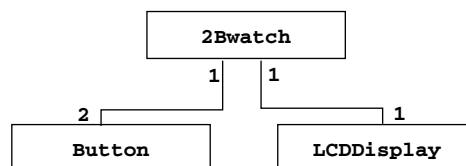


Figure 5-5 An example of multiplicity of associations (UML class diagram). A `2Bwatch` has two buttons and one `LCDDisplay`.

In UML, an association end can have an arbitrary set of integers as a multiplicity. For example, an association could allow only a prime number of links and, thus, would have a multiplicity 1, 2, 3, 5, 7, 11, 13, In practice, however, most of the associations we encounter belong to one of the following three types (see Figure 5-6).

- A **one-to-one association** has a multiplicity 1 on each end. A one-to-one association between two classes (e.g., *PoliceOfficer* and *BadgeNumber*), means that exactly one link exists between instances of each class (e.g., a *PoliceOfficer* has exactly one *BadgeNumber*, and a *BadgeNumber* denotes exactly one *PoliceOfficer*).
- A **one-to-many association** has a multiplicity 1 on one end and 0..n (also represented by a star) or 1..n on the other. A one-to-many association between two classes (e.g., *Person* and *Car*) denotes composition (e.g., a *FireUnit* owns one or more *FireTrucks*, a *FireTruck* is owned exactly by one *FireUnit*).
- A **many-to-many association** has a multiplicity 0..n or 1..n on both ends. A many-to-many association between two classes (e.g., *FieldOfficer* and *IncidentReport*) denotes that an arbitrary number of links can exist between instances of the two classes (e.g., a *FieldOfficer* can write for many *IncidentReport*, an *IncidentReport* can be written by many *FieldOfficers*). This is the most complex type of association.

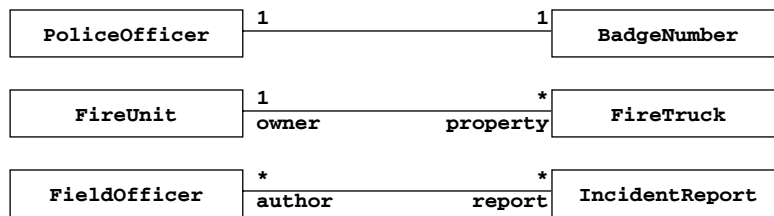


Figure 5-6 Examples of multiplicity (UML class diagram). The association between *Person* and *SocialSecurityNumber* is one-to-one. The association between *Person* and *Car* is one-to-many. The association between *Person* and *Company* is many-to-many.

Adding multiplicity to associations increases the amount of information we capture from the application or the solution domain. Specifying the multiplicity of an association becomes critical when we determine which use cases are needed to manipulate the application domain objects. For example, consider a file system made of *Directory* (ies) and *Files*. A *Directory* can contain any number of *FileSystemElements*. A *FileSystemElement* is an abstract concept that denotes either a *Directory* or a *File*. In case of a strictly hierarchical system, a *FileSystemElement* is part of exactly one *Directory*, which we denote with a one-to-many multiplicity (Figure 5-7).

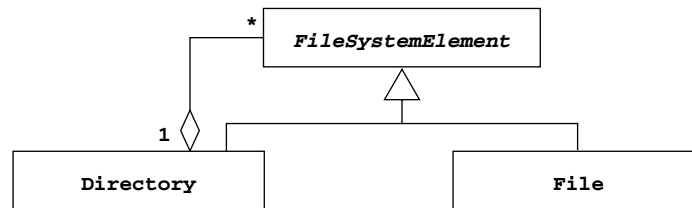


Figure 5-7 Example of a hierarchical file system. A *Directory* can contain any number of *FileSystemElements* (a *FileSystemElement* is either a *File* or a *Directory*). A given *FileSystemElement*, however, is part of exactly one *Directory*.

If, however, a *File* or a *Directory* can be simultaneously part of more than one *Directory*, we need to represent the aggregation of *FileSystemElement* into *Directory* (ies) as a many-to-many association (see Figure 5-8).

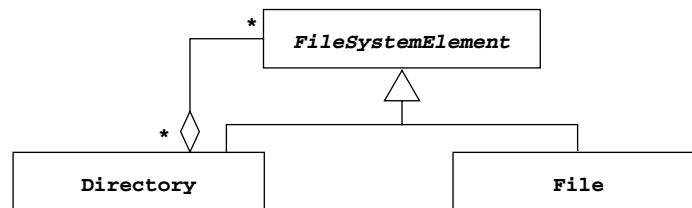


Figure 5-8 Example of a nonhierarchical file system. A *Directory* can contain any number of *FileSystemElements* (a *FileSystemElement* is either a *File* or a *Directory*). A given *FileSystemElement* can be part of many *Directory* (ies).

This discussion may seem to be considering detailed issues that could be left for later activities in the development process. The difference between a hierarchical file system and a nonhierarchical one, however, is also in the functionality it offers. If a system allows a given *File* to be part of multiple *Directory* (ies), we need to define a use case describing how a user adds an existing *File* to existing *Directory* (ies) (e.g., the Unix *link* command or the Macintosh *MakeAlias* menu item). Moreover, use cases removing a *File* from a *Directory* must specify whether the *File* is removed from one *Directory* only or from all *Directory* (ies) that reference them. Note that a many-to-many association can result in a substantially more complex system.

5.3.3 Qualified Associations

Qualification is a technique for reducing multiplicity by using keys. Associations with a 0..1 or 1 multiplicity are easier to understand than associations with a 0..n or 1..n

multiplicity. Often, in the case of a one-to-many association, objects on the “many” side can be distinguished from one another using a name. For example, in a hierarchical file system, each file belongs to exactly one directory. Each file is uniquely identified by a name in the context of a directory. Many files can have the same name in the context of the file system; however, two files cannot share the same name within the same directory. Without qualification (see top of Figure 5-9), the association between `Directory` and `File` has a one multiplicity on the `Directory` side and a zero-to-many multiplicity on the `File` side. We reduce the multiplicity on the `File` side by using the `filename` attribute as a key, also called a **qualifier** (see top of Figure 5-9). The relationship between `Directory` and `File` is called a **qualified association**.

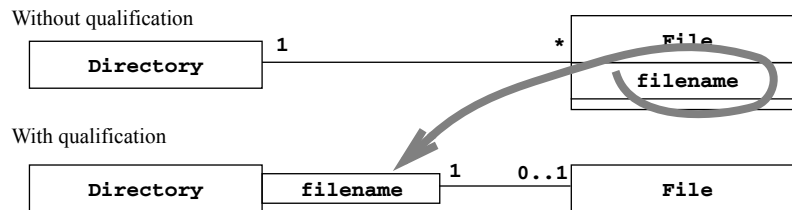


Figure 5-9 Example of how a qualified association reduces multiplicity (UML class diagram). Adding a qualifier clarifies the class diagram and increases the conveyed information. In this case, the model including the qualification denotes that the name of a file is unique within a directory.

Reducing multiplicity is always preferable, as the model becomes clearer and fewer cases have to be taken into account. Developers should examine each association that has a one-to-many or many-to-many multiplicity and check if a qualifier can be added. Often, these associations can be qualified with an attribute of the target class, (e.g., the `filename` attribute in Figure 5-9).

5.3.4 Generalization

As we saw in Chapter 2, *Modeling with UML*, **generalization** enables us to organize concepts into hierarchies. At the top of the hierarchy is a general concept (e.g., an `Incident`, Figure 5-10), and at the bottom of the hierarchy are the most specialized concepts (e.g., `CatInTree`, `TrafficAccident`, `BuildingFire`, `EarthQuake`, `ChemicalLeak`). There may be any number of intermediate levels in between, covering more-or-less generalized concepts (e.g., `LowPriorityIncident`, `Emergency`, `Disaster`). Such hierarchies allow us to refer to many concepts precisely. When we use the term `Incident`, we mean all instances of all types of `Incidents`. When we use the term `Emergency`, we only refer to an `Incident` that requires an immediate response. This view of generalization stems from modeling.

In an object-oriented programming language, **inheritance** is a reusability technique. If a class `Child` inherits from a class `Parent`, all the attributes and methods available on `Parent` are

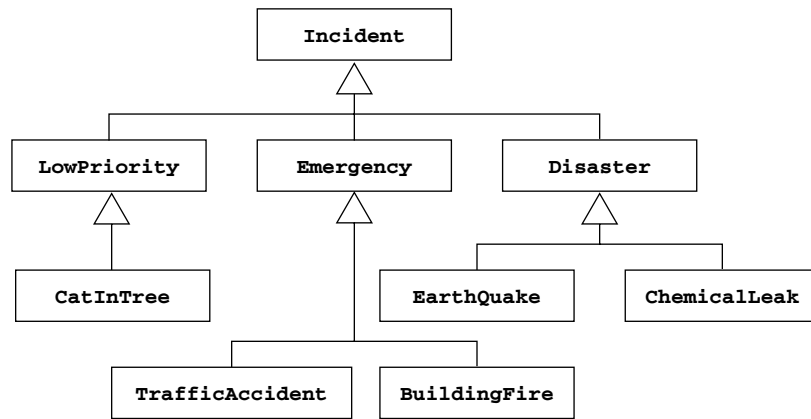


Figure 5-10 An example of a generalization hierarchy (UML class diagram). The root of the hierarchy represents the most general concept, whereas the leaves nodes represent the most specialized concepts.

automatically available on `Child`. The class `Child` may add additional methods or override inherited methods, thus refining the class `Parent`. As in the case of generalization, the class at the top of the hierarchy tends to be the most general one, whereas the leaves tend to be the most specialized.

Generalization and inheritance are closely related concepts. They are, however, not identical. Inheritance is a mechanism for reusing attributes and behavior, even when the classes involved in the inheritance do not have a generalization relationship. For example, it is possible to implement a `Set` collection by refining a `Hashtable` (although this might not be a good solution for implementing `Sets`). Nevertheless, a `Set` is not more specialized a concept than a `Hashtable`, nor is a `Hashtable` a generalization of a `Set`.

During analysis, we only focus on the organization of concepts into generalization relationships and should not be interpreted in terms of reuse. The inheritance hierarchy, although initially derived from the generalization hierarchy, will be restructured during object design.

5.4 Analysis Activities: From Use Cases to Objects

In this section, we describe the activities that transform the use cases and scenarios produced during requirements elicitation into an analysis model. Analysis activities include:

- identifying entity objects (Section 5.4.1)
- identifying boundary objects (Section 5.4.2)
- identifying control objects (Section 5.4.3)
- mapping use cases to objects (Section 5.4.4)
- identifying associations among objects (Section 5.4.5)



- identifying object attributes (Section 5.4.6)
- modeling nontrivial behavior with statecharts (Section 5.4.7)
- modeling generalization relationships (Section 5.4.8)
- reviewing the analysis model (Section 5.4.9)

We illustrate each activity by focusing on the `ReportEmergency` use case of `FRIEND` described in Chapter 4, *Requirements Elicitation*. These activities are guided by heuristics. The quality of their outcome depends on the experience of the developer in applying these heuristics and methods. The methods and heuristics presented in this section are adapted from [De Marco, 1978], [Jacobson et al., 1999], [Rumbaugh et al., 1991], and [Wirfs-Brock et al., 1990].

5.4.1 Identifying Entity Objects

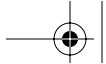
Participating objects (see Section 4.4.6) form the basis of the analysis model. As described in Chapter 4, *Requirements Elicitation*, participating objects are found by examining each use case and identifying candidate objects. Natural language analysis [Abbott, 1983] is an intuitive set of heuristics for identifying objects, attributes, and associations from a system specification. Abbott's heuristics maps parts of speech (e.g., nouns, having verbs, being verbs, adjectives) to model components (e.g., objects, operations, inheritance relationships, classes). Table 5-1 provides examples of such mappings by examining the `ReportEmergency` use case Figure 5-11.

Table 5-1 Abbott's heuristics for mapping parts of speech to model components [Abbott, 1983]

Part of speech	Model component	Examples
Proper noun	Object	Alice
Common noun	Class	FieldOfficer
Doing verb	Operation	Creates, submits, selects
Being verb	Inheritance	Is a kind of, is one of either
Having verb	Aggregation	Has, consists of, includes
Modal verb	Constraints	Must be
Adjective	Attribute	Incident description

Natural language analysis has the advantage of focusing on the users' terms. However, it suffers from several limitations. First, the quality of the object model depends highly on the style of writing of the analyst (e.g., consistency of terms used, verbification of nouns). Natural language is an imprecise tool, and an object model derived literally from text risks being imprecise. Developers can address this limitation by rephrasing and clarifying the system specification as they identify and standardize objects and terms. A second limitation of natural language analysis is that there are many more nouns than relevant classes. Many nouns





correspond to attributes or synonyms for other nouns. Sorting through all the nouns for a large system specification is a time-consuming activity. In general, Abbott's heuristics work well for generating a list of initial candidate objects from short descriptions, such as the flow of events of a scenario or a use case. The following heuristics can be used in conjunction with Abbott's rules:

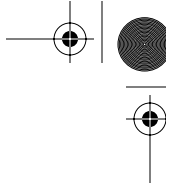
Heuristics for identifying entity objects

- terms that developers or users need to clarify in order to understand the use case
- recurring nouns in the use cases (e.g., *Incident*)
- real-world entities that the system needs to keep track of (e.g., *FieldOfficer*, *Dispatcher*, *Resource*)
- real-world activities that the system needs to keep track of (e.g., *EmergencyOperationsPlan*)
- use cases (e.g., *ReportEmergency*)
- data sources or sinks (e.g., *Printer*)
- *always* use the user's terms

Developers name and briefly describe the objects, their attributes, and their responsibilities as they are identified. Uniquely naming objects promotes a standard terminology. Describing objects, even briefly, allows developers to clarify the concepts they use and avoid misunderstandings (e.g., using one object for two different but related concepts). Developers need not, however, spend a lot of time detailing objects or attributes given that the analysis model is still in flux. Developers should document attributes and responsibilities if they are obvious. A tentative name and a brief description for each object is sufficient otherwise. There will be plenty of iterations during which objects can be revised. However, once the analysis model is stable, the description of each object should be as detailed as necessary (see Section 5.4.9).

For example, after a first examination of the *ReportEmergency* use case (Figure 5-11), we use application domain knowledge and interviews with the users to identify the objects *Dispatcher*, *EmergencyReport*, *FieldOfficer*, and *Incident*. Note that the *EmergencyReport* object is not mentioned explicitly by name in the *ReportEmergency* use case. Step 3 of the use case refers to the emergency report as the "information submitted by the *FieldOfficer*." After review with the client, we discover that this information is usually referred to as the emergency report and decide to name the corresponding object *EmergencyReport*.





<i>Use case name</i>	ReportEmergency
<i>Entry condition</i>	1. The <code>FieldOfficer</code> activates the “Report Emergency” function of her terminal.
<i>Flow of events</i>	2. <code>FRIEND</code> responds by presenting a form to the officer. The form includes an emergency type menu (General emergency, fire, transportation), a location, incident description, resource request, and hazardous material fields. 3. The <code>FieldOfficer</code> fills the form, by specifying minimally the emergency type and description fields. The <code>FieldOfficer</code> may also describes possible responses to the emergency situation and request specific resources. Once the form is completed, the <code>FieldOfficer</code> submits the form by pressing the “Send Report” button, at which point, the <code>Dispatcher</code> is notified. 4. The <code>Dispatcher</code> reviews the information submitted by the <code>FieldOfficer</code> and creates an <code>Incident</code> in the database by invoking the <code>OpenIncident</code> use case. All the information contained in the <code>FieldOfficer</code>’s form is automatically included in the incident. The <code>Dispatcher</code> selects a response by allocating resources to the incident (with the <code>AllocateResources</code> use case) and acknowledges the emergency report by sending a <code>FRIENDgram</code> to the <code>FieldOfficer</code>.
<i>Exit condition</i>	5. The <code>FieldOfficer</code> receives the acknowledgment and the selected response.

Figure 5-11 An example of use case: `ReportEmergency`.

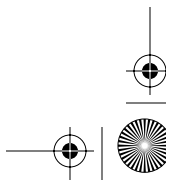
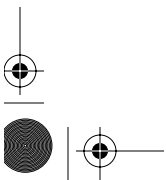
The definition of entity objects leads to the initial analysis model described in Table 5-2.

Note that the above object model is far from being a complete description of the system implementing the `ReportEmergency` use case. In the next section, we describe the identification of boundary objects.

5.4.2 Identifying Boundary Objects

Boundary objects represent the system interface with the actors. In each use case, each actor interacts with at least one boundary object. The boundary object collects the information from the actor and translates it into an interface neutral form that can be used by the entity objects and also by the control objects.

Boundary objects model the user interface at a coarse level. They do not describe in detail the visual aspects of the user interface. For example, boundary objects such as “button” or “menu item” are too detailed. First, developers can discuss user interface details more easily with sketches and mock-ups. Second, the design of the user interface design will continue to evolve as a consequence of usability tests, even after the functional specification of the system



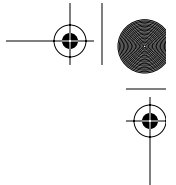
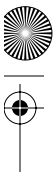


Table 5-2 Entity objects for the ReportEmergency use case

Dispatcher	Police officer who manages Incidents. A Dispatcher opens, documents, and closes Incidents in response to Emergency Reports and other communication with FieldOfficers. Dispatchers are identified by badge numbers.
EmergencyReport	Initial report about an Incident from a FieldOfficer to a Dispatcher. An EmergencyReport usually triggers the creation of an Incident by the Dispatcher. An EmergencyReport is composed of a emergency level, a type (fire, road accident, or other), a location, and a description.
FieldOfficer	Police or fire officer on duty. A FieldOfficer can be allocated to, at most, one Incident at a time. FieldOfficers are identified by badge numbers.
Incident	Situation requiring attention from a FieldOfficer. An Incident may be reported in the system by a FieldOfficer or anybody else external to the system. An Incident is composed of a description, a response, a status (open, closed, documented), a location, and a number of FieldOfficers.

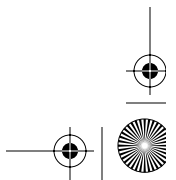
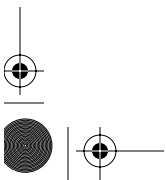
becomes stable. Updating the analysis model every time a visual change is made to the interface is time consuming and does not yield any substantial benefit.

Heuristics for identifying boundary objects

- Identify forms and windows the users needs to enter data into the system (e.g., EmergencyReportForm, ReportEmergencyButtonForm).
- Identify notices and messages the system uses to respond to the user (e.g., AcknowledgmentNotice).
- Do not model the visual aspects of the interface with boundary objects (user mock-ups are better suited for that).
- *Always* use the user's terms for describing interfaces as opposed to terms from the implementation technology.

We find the following boundary objects by examining the ReportEmergency use case (Table 5-3).

Note that the IncidentForm is not explicitly mentioned anywhere in the ReportEmergency use case. We identified this object by observing that the Dispatcher needs an interface to view the emergency report submitted by the FieldOfficer and to send back an acknowledgment. The terms used for describing the boundary objects in the analysis model should follow the user terminology, even if it is tempting to use terms from the implementation domain.



**Table 5-3** Boundary objects for the ReportEmergency use case

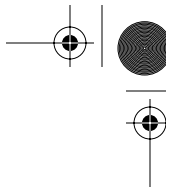
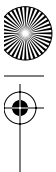
AcknowledgmentNotice	Notice used for displaying the Dispatcher's acknowledgment to the FieldOfficer.
DispatcherStation	Computer used by the Dispatcher.
ReportEmergencyButton	Button used by a FieldOfficer to initiate the ReportEmergency use case.
ReportEmergencyForm 	Form used for the input of the ReportEmergencyForm. This form is presented to the FieldOfficer on the FieldOfficerStation when the "Report Emergency" function is selected. The EmergencyReportForm contains fields for specifying all attributes of an emergency report and a button (or other control) for submitting the form once it is completed.
FieldOfficerStation	Mobile computer used by the FieldOfficer.
IncidentForm	Form used for the creation of Incidents. This form is presented to the Dispatcher on the DispatcherStation when the EmergencyReport is received. The Dispatcher also uses this form to allocate resources and to acknowledge the FieldOfficer's report.

We have made progress toward describing the system. We now have included the interface between the actor and the system. We are, however, still missing some significant pieces of the description, such as the order in which the interactions between the actors and the system occur. In the next section, we describe the identification of control objects.

5.4.3 Identifying Control Objects

Control objects are responsible for coordinating boundary and entity objects. Control objects usually do not have a concrete counterpart in the real world. There is often a close relationship between a use case and a control object. A control object is usually created at the beginning of a use case and ceases to exist at its end. It is responsible for collecting information from the boundary objects and dispatching it to entity objects. For example, control objects describe the behavior associated with the sequencing of forms, undo and history queues, and dispatching information in a distributed system.



**Heuristics for identifying control objects**

- Identify one control object per use case or more if the use case is complex and if it can be divided into shorter flows of events.
- Identify one control object per actor in the use case.
- The life span of a control object should be extent of the use case or the extent of a user session. If it is difficult to identify the beginning and the end of a control object activation, the corresponding use case may not have a well-defined entry and exit condition.

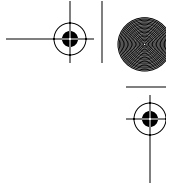
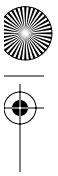
Initially, we model the control flow of the `ReportEmergency` use case with a control object for each actor; `ReportEmergencyControl` for the `FieldOfficer` and `ManageEmergencyControl` for the `Dispatcher`, respectively (Table 5-4).

The decision to model the control flow of the `ReportEmergency` use case with two control objects stems from the knowledge that the `FieldOfficerStation` and the `DispatcherStation` are actually two subsystems communicating over an asynchronous link. This decision could have been postponed until the system design activity. On the other hand, making this concept visible in the analysis model allows us to focus on such exception behavior as the loss of communication between both stations.

In modeling the `ReportEmergency` use case, we modeled the same functionality by using entity, boundary, and control objects. By shifting from the event-flow perspective to a structural perspective, we increased the level of detail of the description and selected standard terms to

Table 5-4 Control objects for the `ReportEmergency` use case

<code>ReportEmergencyControl</code>	Manages the report emergency reporting function on the <code>FieldOfficerStation</code> . This object is created when the <code>FieldOfficer</code> selects the “Report Emergency” button. It then creates an <code>EmergencyReportForm</code> and presents it to the <code>FieldOfficer</code> . After submission of the form, this object then collects the information from the form, creates an <code>EmergencyReport</code> , and forwards it to the <code>Dispatcher</code> . The control object then waits for an acknowledgment to come back from the <code>DispatcherStation</code> . When the acknowledgment is received, the <code>ReportEmergencyControl</code> object creates an <code>AcknowledgmentNotice</code> and displays it to the <code>FieldOfficer</code> .
<code>ManageEmergencyControl</code>	Manages the report emergency reporting function on the <code>DispatcherStation</code> . This object is created when an <code>EmergencyReport</code> is received. It then creates an <code>IncidentForm</code> and displays it to the <code>Dispatcher</code> . Once the <code>Dispatcher</code> has created an <code>Incident</code> , allocated <code>Resources</code> , and submitted an acknowledgment, <code>ManageEmergencyControl</code> forwards the acknowledgment to the <code>FieldOfficerStation</code> .



refer to the main entities of the application domain and the system. In the next section, we construct a sequence diagram using the `ReportEmergency` use case and the objects we discovered to ensure the completeness of our model.

5.4.4 Modeling Interactions Between Objects: Sequence Diagrams

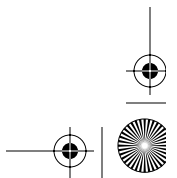
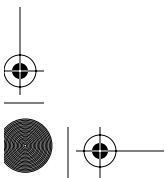
A sequence diagram ties use cases with objects. It shows how the behavior of a use case (or scenario) is distributed among its participating objects. Sequence diagrams are usually not a good medium for communication with the user. They represent, however, another shift in perspective and allow the developers to find missing objects or grey areas in the system specification.

In this section, we model the sequence of interactions among objects needed to realize the use case. Figures 5-12–5-14 are sequence diagrams associated with the `ReportEmergency` use case. The columns of a sequence diagram represent the objects that participate in the use case. The leftmost column is the actor who initiates the use case. Horizontal arrows across columns represent messages, or stimuli, which are sent from one object to the other. Time proceeds vertically from top to bottom. For example, the arrow in Figure 5-12 represents the `press` message sent by a `FieldOfficer` to an `ReportEmergencyButton`. The receipt of a message triggers the activation of an operation. The activation is represented by a rectangle from which other messages can originate. The length of the rectangle represents the time the operation is active. In Figure 5-12, the operation triggered by the `press` message sends a `create` message to the `ReportEmergencyControl` class. An operation can be thought of as a service that the object provides to other objects.

Heuristics for drawing sequence diagrams

- The first column should correspond to the actor who initiated the use case.
- The second column should be an boundary object (that the actor used to initiate the use case).
- The third column should be the control object that manages the rest of the use case.
- Control objects are created by boundary objects initiating use cases.
- Boundary objects are created by control objects.
- Entity objects are accessed by control and boundary objects.
- Entity objects *never* access boundary or control objects, this makes it easier to share entity objects across use cases.

In general, the second column of a sequence diagram represents the boundary object with which the actor interacts to initiate the use case (e.g., `ReportEmergencyButton`). The third column is a control object that manages the rest of the use case (e.g., `ReportEmergencyControl`). From then on, the control object creates other boundary objects and may interact with other control objects as well (in this case, the `ManageEmergencyControl` object).



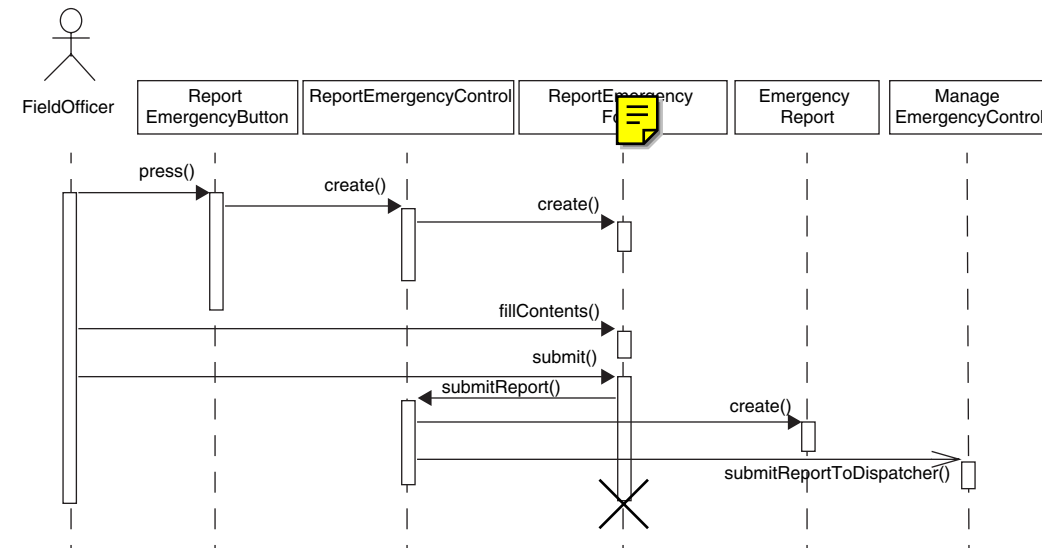


Figure 5-12 Sequence diagram for the ReportEmergency use case (initiation from the FieldOfficerStation side).

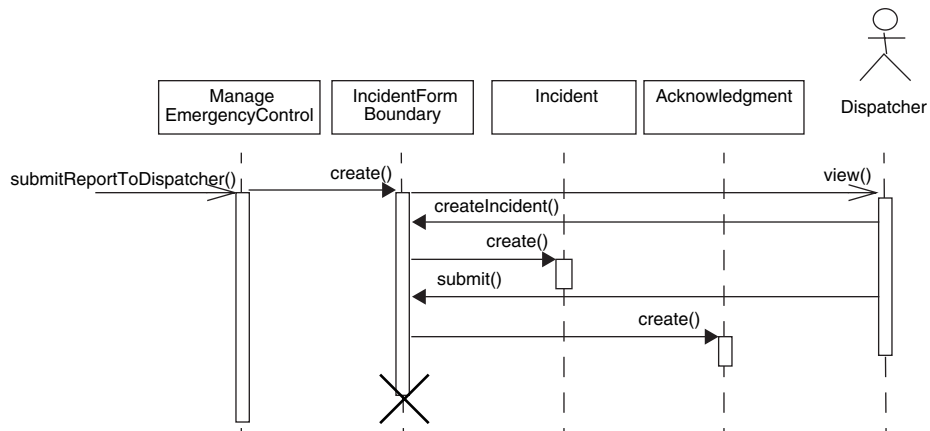


Figure 5-13 Sequence diagram for the ReportEmergency use case (DispatcherStation).

In Figure 5-13, we discover the entity object Acknowledgment that we forgot during our initial examination of the ReportEmergency use case (in Table 5-2). The Acknowledgment object is different from an AcknowledgmentNotice: It holds the information associated with an Acknowledgment and is created before the AcknowledgmentNotice boundary object.

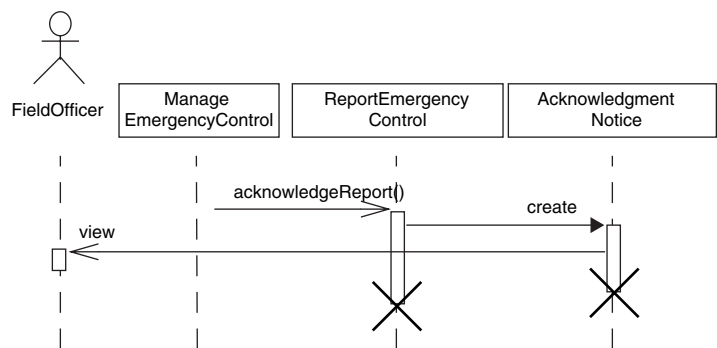
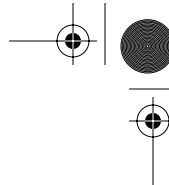


Figure 5-14 Sequence diagram for the `ReportEmergency` use case (acknowledgment on the `FieldOfficerStation`).

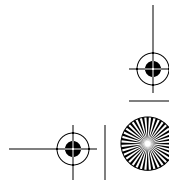
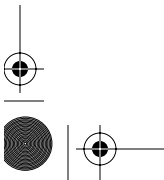
When describing the `Acknowledgment` object, we also realize that the original `ReportEmergency` use case (described in Figure 5-11) is incomplete. It only mentions the existence of an `Acknowledgment` and does not describe the information associated with it. In this case, developers need clarification from the client to define what information needs to appear in the `Acknowledgment`. After obtaining such clarification, the `Acknowledgment` object is added to the analysis model (Table 5-5) and the `ReportEmergency` use case is clarified to include the additional information (Figure 5-15).

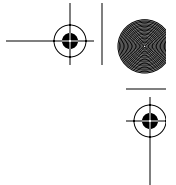
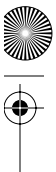
Table 5-5 `Acknowledgment` object for the `ReportEmergency` use case

Acknowledgment	Response of a dispatcher to a <code>FieldOfficer</code> 's <code>EmergencyReport</code> . By sending an <code>Acknowledgment</code> , the Dispatcher communicates to the <code>FieldOfficer</code> that she has received the <code>EmergencyReport</code> , created an Incident, and assigned resources to it. The <code>Acknowledgment</code> contains the assigned resources and their estimated arrival time.
-----------------------	--

By constructing sequence diagrams, we not only model the order of the interaction among the objects, we also distribute the behavior of the use case. In other terms, we assign to each object responsibilities in the form of a set of operations. These operations can be shared by any use case in which a given object participates. Note that the definition of an object that is shared across two or more use cases should be identical. In other terms, if an operation appears in more than one sequence diagram, its behavior should be the same.

Sharing operations across use cases allows developers to remove redundancies in the system specification and to improve its consistency. Note that clarity should always be given precedence to eliminating redundancy. Fragmenting behavior across many operations unnecessarily complicates the system specification.





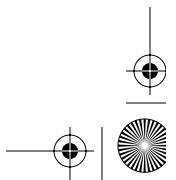
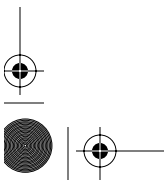
<i>Use case name</i>	ReportEmergency
<i>Entry condition</i>	1. The <code>FieldOfficer</code> activates the “Report Emergency” function of her terminal.
<i>Flow of events</i>	2. <code>FRIEND</code> responds by presenting a form to the officer. The form includes an emergency type menu (General emergency, fire, transportation), a location, incident description, resource request, and hazardous material fields. 3. The <code>FieldOfficer</code> fills the form, by specifying minimally the emergency type and description fields. The <code>FieldOfficer</code> may also describes possible responses to the emergency situation and request specific resources. Once the form is completed, the <code>FieldOfficer</code> submits the form by pressing the “Send Report” button, at which point the <code>Dispatcher</code> is notified. 4. The <code>Dispatcher</code> reviews the information submitted by the <code>FieldOfficer</code> and creates an <code>Incident</code> in the database by invoking the <code>OpenIncident</code> use case. All the information contained in the <code>FieldOfficer</code>’s form is automatically included in the incident. The <code>Dispatcher</code> selects a response by allocating resources to the incident (with the <code>AllocateResources</code> use case) and acknowledges the emergency report by sending a <code>FRIENDgram</code> to the <code>FieldOfficer</code>. The Acknowledgment indicates to the FieldOfficer that the EmergencyReport was received, an Incident created, and resources allocated to the Incident. The Acknowledgment includes the resources (e.g., a fire truck) and their estimated arrival time.
<i>Exit condition</i>	5. The <code>FieldOfficer</code> receives the <code>Acknowledgment</code> and the selected response.

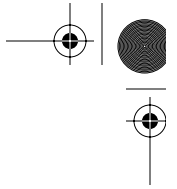
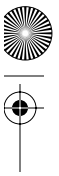
Figure 5-15 Refined `ReportEmergency` use case. The discovery and addition of the `Acknowledgment` object to the analysis model revealed that the original `ReportEmergency` use case did not accurately describe the information associated with `Acknowledgments`. The refinements are indicated in **boldface**.

In analysis, sequence diagrams are used to help identify new participating objects and missing behavior. They focus on high-level behavior, and, thus, implementation issues such as performance should not be addressed at this point. Given that building interaction diagrams can be time consuming, developers should focus on problematic or underspecified functionality first. Drawing interaction diagrams for parts of the system that are simple or well defined is not a good investment of analysis resources.

5.4.5 Identifying Associations

Whereas sequence diagrams allow developers to represent interactions among objects over time, class diagrams allow developers to describe the spatial connectivity of objects. We described the UML class diagram notation in Chapter 2, *Modeling with UML*, and use it throughout the book to represent various project artifacts (e.g., activities, deliverables). In this section, we discuss the use of class diagrams for representing associations among objects. In Section 5.4.6, we discuss the use of class diagrams for representing object attributes.





An association shows a relationship between two or more classes. For example, a `FieldOfficer` writes an `EmergencyReport` (see Figure 5-16). Identifying associations has two advantages. First, it clarifies the analysis model by making relationships between objects explicit (e.g., an `EmergencyReport` can be created by a `FieldOfficer` or a `Dispatcher`). Second, it enables the developer to discover boundary cases associated with links. Boundary cases are exceptions that need to be clarified in the model. For example, it is intuitive to assume that most `EmergencyReports` are written by one `FieldOfficer`. However, should the system support `EmergencyReports` written by more than one? Should the system allow for anonymous `EmergencyReports`? Those questions should be investigated during analysis by using domain knowledge.

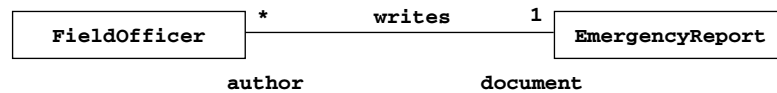


Figure 5-16 An example of association between the `EmergencyReport` and the `FieldOfficer` classes.

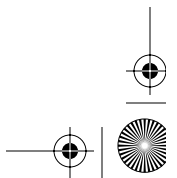
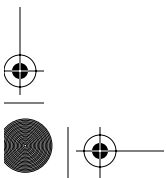
Associations have several properties:

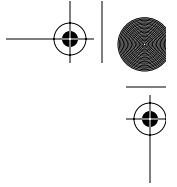
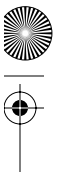
- A **name**, to describe the association between the two classes (e.g., `Writes` in Figure 5-16). Association names are optional and need not be unique globally.
- A **role** at each end, identifying the function of each class with respect to the associations (e.g., `author` is the role played by `FieldOfficer` in the `Writes` association).
- A **multiplicity** at each end, identifying the possible number of instances (e.g., `*` indicates a `FieldOfficer` may write zero or more `EmergencyReports`, whereas `1` indicates that each `EmergencyReport` has exactly one `FieldOfficer` as author).

Initially, the associations between entity objects are the most important, as they reveal more information about the application domain. According to Abbott's heuristics (see Table 5-1), associations can be identified by examining verbs and verb phrases denoting a state (e.g., *has*, *is part of*, *manages*, *reports to*, *is triggered by*, *is contained in*, *talks to*, *includes*). Every association should be named and roles assigned to each end.

Heuristics for identifying associations

- Examine verb phrases.
- Name associations and roles precisely.
- Use qualifiers as often as possible to identify namespaces and key attributes.
- Eliminate any association that can be derived from other associations.
- Do not worry about multiplicity until the set of associations is stable.
- Avoid ravioli models: Too many associations make a model unreadable.





The object model will initially include too many associations if developers include all associations identified after examining verb phrases. In Figure 5-17, for example, we identified two relationships: the first between an Incident and the EmergencyReport that triggered its creation; the second between the Incident and the reporting FieldOfficer. Given that the EmergencyReport and FieldOfficer already have an association modeling authorship, the association between Incident and FieldOfficer is not necessary. Adding unnecessary associations complicates the model, leading to incomprehensible “ravioli models” and redundant information.

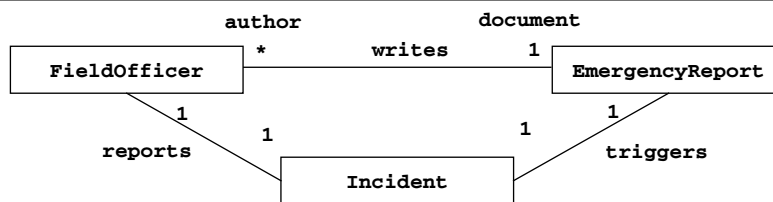
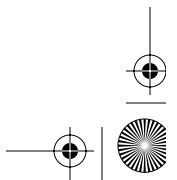
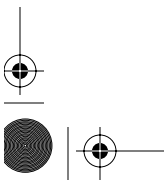


Figure 5-17 Eliminating redundant association. The receipt of an EmergencyReport triggers the creation of an Incident by a Dispatcher. Given that the EmergencyReport has an association with the FieldOfficer that wrote it, it is not necessary to keep an association between FieldOfficer and Incident.

Most entity objects have an identifying characteristic used by the actors to access them. FieldOfficers and Dispatchers have a badge number. Incidents and Reports are assigned numbers and are archived by date. Once the analysis model includes most classes and associations, the developers should go through each class and check how it is identified by the actors and in which context. For example, are FieldOfficer badge numbers unique across the universe? Across a city? A police station? If they are unique across cities, can the FRIEND system know about FieldOfficers from more than one city? This approach can be formalized by examining each individual class and identifying the sequence of associations that need to be traversed to access a specific instance of that class.

5.4.6 Identifying Attributes

Attributes are properties of individual objects. For example, an EmergencyReport, as described in Table 5-2, has an emergency type, a location, and a description property (see Figure 5-18). These are entered by a FieldOfficer when she reports an emergency and are subsequently tracked by the system. When identifying properties of objects, only the attributes relevant to the system should be considered. For example, each FieldOfficer has a social security number that is not relevant to the emergency information system. Instead, FieldOfficers are identified by badge number, represented by the badgeNumber property.



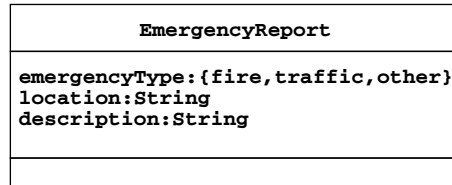
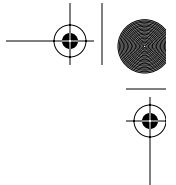
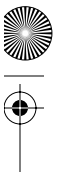


Figure 5-18 Attributes of the EmergencyReport class.

Properties that are represented by objects are not attributes. For example, every EmergencyReport has an author represented by an association to the FieldOfficer class. Developers should identify as many associations as possible before identifying attributes to avoid confusing attributes and objects. Attributes have:

- a **name** identifying them within an object. For example, an EmergencyReport may have a reportType attribute and an emergencyType attribute. The reportType describes the kind of report being filed (e.g., initial report, request for resource, final report). The emergencyType describes the type of emergency (e.g., fire, traffic, other). To avoid confusion, these attributes should not be both called type.
- A brief **description**.
- A **type** describing the legal values it can take. For example, the description attribute of an EmergencyReport is a string. The emergencyType attribute is an enumeration that can take one of three values: fire, traffic, other.

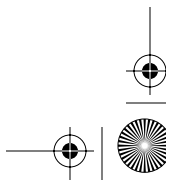
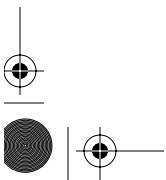
Attributes can be identified using Abbott's heuristics (see Table 5-1). In particular, noun phrases followed by a possessive phrases (e.g., the description of an emergency) or an adjective phrase (e.g., the emergency description) should be examined. In the case of entity objects, any property that needs to be stored by the system is a candidate attribute.

Note that attributes represent the least stable part of the object model. Often, attributes are discovered or added late in the development when the system is evaluated by the users. Unless the added attributes are associated with additional functionality, the added attributes do not

Heuristics for identifying attributes^a

- Examine possessive phrases.
- Represent stored state as attributes of entity object.
- Describe each attribute.
- Do not represent an attribute as an object, use an association instead (see Section 5.4.5).
- Do not waste time describing fine details before the object structure is stable.

a. Adapted from [Rumbaugh et al., 1991].



entail major changes in the object (and system) structure. For these reasons, the developers need not spend excessive resources in identifying and detailing attributes representing less important aspects of the system. These attributes can be added later when the analysis model or the user interface sketches are validated.

5.4.7 Modeling the Nontrivial Behavior of Individual Objects

Sequence diagrams are used to distribute behavior across objects and to identify operations. Sequence diagrams represent the behavior of the system from the perspective of a single use case. Statechart diagrams represent behavior from the perspective of a single object. Viewing behavior from the perspective of each object enables the developer, on the one hand, to identify missing use cases, and, on the other hand, to build a more-formal description of the behavior of the object. Note that it is not necessary to build statecharts for every class in the system. Only the statecharts of objects with an extended lifespan and nontrivial behavior are worth constructing.

Figure 5-19 displays a statechart for the *Incident* class. The examination of this statechart may help the developer check if there are use cases for documenting, closing, and archiving *Incidents*. Note that Figure 5-19 is a high-level statechart and does not model the state changes an *Incident* goes through while it is active (e.g., when resources are assigned to it). Such behavior can be modeled by associating a nested statechart with the *Active* state.

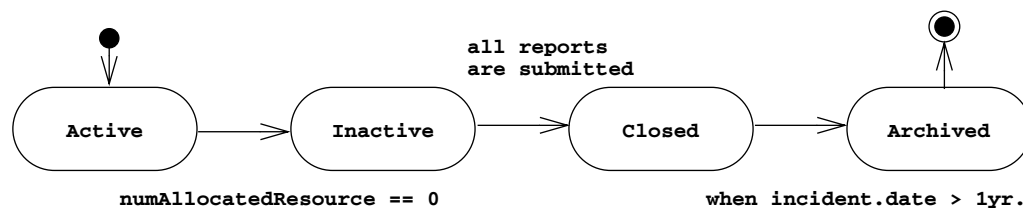


Figure 5-19 UML statechart for *Incident*.

5.4.8 Modeling Generalization Relationships Between Objects

Generalization is used to eliminate redundancy from the analysis model. If two or more classes share attributes or behavior, the similarities are consolidated into a superclass. For example, *Dispatchers* and *FieldOfficers* both have a *badgeNumber* attribute that serves to identify them within a city. *FieldOfficers* and *Dispatchers* are both *PoliceOfficers* who are assigned different functions. To model explicitly this similarity, we introduce an abstract *PoliceOfficer* class from which the *FieldOfficer* and *Dispatcher* classes inherit (see Figure 5-20).

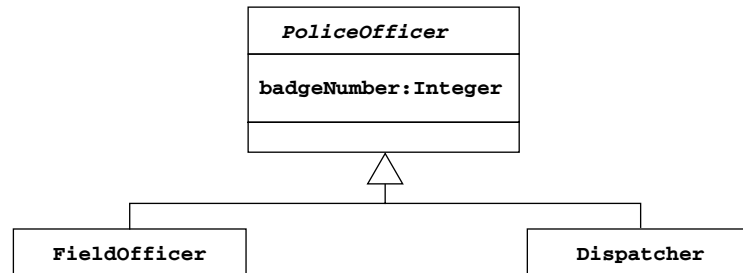


Figure 5-20 An example of inheritance relationship (UML class diagram).

5.4.9 Reviewing the Analysis Model

The analysis model is built incrementally and iteratively. The analysis model is seldom correct or even complete on the first pass. Several iterations with the client and the user are necessary before the analysis model converges toward a correct specification usable by the developers for proceeding to design and implementation. For example, an omission discovered during analysis will lead to adding or extending a use case in the system specification, which may lead to eliciting more information from the user.

Once the analysis model becomes stable (i.e., when the number of changes to the model are minimal and the scope of the changes localized), the analysis model is reviewed, first by the developers (i.e., internal reviews), then jointly by the developers and the client. The goal of the review is to make sure that the system specification is correct, complete, consistent, and realistic. Note that developers should be prepared to discover errors downstream and make changes to the specification. It is, however, a worthwhile investment to catch as many requirements errors upstream. The review can be facilitated by a checklist or a list of questions. Below are example questions adapted from [Jacobson et al., 1999] and [Rumbaugh et al., 1991].

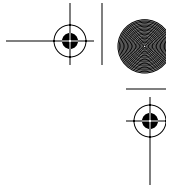
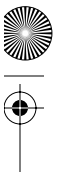
The following questions should be asked to ensure that the model is *correct*:

- Is the glossary of entity objects understandable by the user?
- Do abstract classes correspond to user-level concepts?
- Are all descriptions in accordance with the users' definitions?
- Do all entity and boundary objects have meaningful noun phrases as names?
- Do all use cases and control objects have meaningful verb phrases as names?
- Are all error cases described and handled?
- Are the start-up and the shut-down phases of the system described?
- Are the administration functions of the system described?

The following questions should be asked to ensure that the model is *complete*:

- For each object: Is it needed by any use case? In which use case is it created? modified? destroyed? Can it be accessed from an boundary object?





- For each attribute: When is it set? What is its type? Should it be a qualifier?
- For each association: When is it traversed? Why was the specific multiplicity chosen? Can associations with one-to-many and many-to-many multiplicities be qualified?
- For each control object: Does it have the necessary associations to access the objects participating in its corresponding use case?

The following questions should be asked to ensure that the model is *consistent*:

- Are there multiple classes or use cases with the same name?
- Do entities (e.g., use cases, classes, attributes) with similar names denote similar phenomena?
- Are all entities described at the same level of detail?
- Are there objects with similar attributes and associations that are not in the same generalization hierarchy?

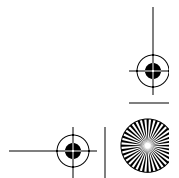
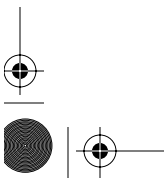
The following questions should be asked to ensure that the system described by the analysis model is *realistic*:

- Are there any novel features in the system? Were there any studies or prototypes built to ensure their feasibility?
- Can the performance and reliability requirements be met? Were these requirements verified by any prototypes running on the selected hardware?

5.4.10 Analysis Summary

The requirements activity is highly iterative and incremental. Chunks of functionality are sketched and proposed to the users and the client. The client adds additional requirements, criticizes existing functionality, and modifies existing requirements. The developers investigate nonfunctional requirements through prototyping and technology studies and challenge each proposed requirement. Initially, requirements elicitation resembles a brainstorming activity. As the description of the system grows and the requirements become more concrete, developers need to extend and modify the analysis model in a more orderly manner in order to manage the complexity of information.

Figure 5-21 depicts a typical sequence of the analysis activities we described in this chapter. The users, developers, and client are involved in the *Define use cases* and develop an initial use case model. They identify a number of concepts and build a glossary of participating objects. The developers then classify these objects into entity, boundary, and control objects (in *Define entity objects*, Section 5.4.1, *Define boundary objects*, Section 5.4.2, and *Define control objects*, Section 5.4.3). These activities occur in a tight loop until most of the functionality of the system has been identified as use cases with names and brief descriptions. Then, the developers construct sequence diagrams to identify any missing objects (*Define interactions*, Section 5.4.4). Once all entity objects have been named and briefly described, the analysis model should remain fairly stable as it is refined.



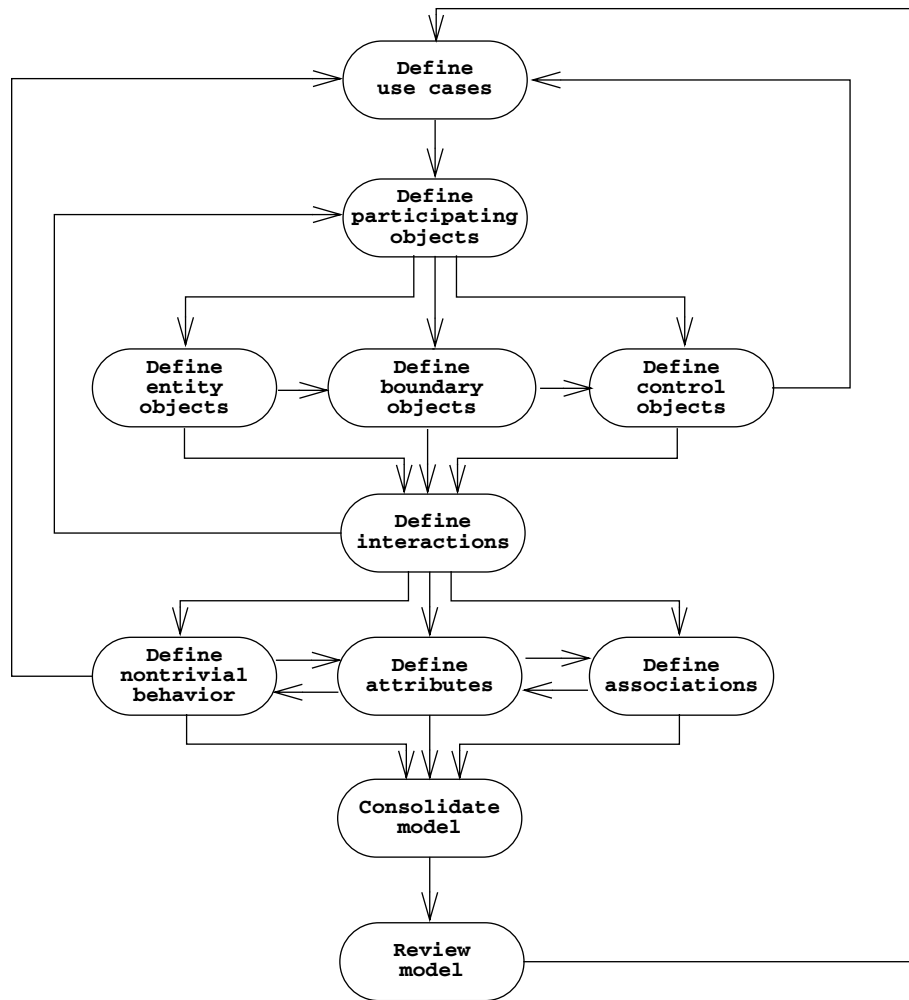


Figure 5-21 Analysis activities (UML activities diagram).

Define interesting behavior (Section 5.4.7), *Define attributes* (Section 5.4.6), and *Define associations* (Section 5.4.5) constitute the refinement of the analysis model. These three activities represent a tight loop during which the state of the objects and their associations are extracted from the sequence diagrams and detailed. The use cases are then modified to account for any changes in functionality. This phase may lead to the identification of an additional chunk of functionality in the form of additional use cases. The overall process is then repeated incrementally for these new use cases.



During *Consolidate model* (Section 5.4.8), the developers solidify the model by introducing qualifiers, generalization relationships, and suppressing redundancies. During *Review model* (Section 5.4.9), the client, users, and developers examine the model for correctness, consistency, completeness, and realism. The project schedule should plan for multiple reviews to ensure high quality of the requirements and to allow space for learning the requirements activity. However, once the model reaches the point where most modifications are cosmetic, system design should proceed. There will be a point during requirements where no more problems can be anticipated without further information from prototyping, usability studies, technology surveys, or system design. Getting every detail right becomes a wasteful exercise: Some of these details will become irrelevant by the next change. Management should recognize this point and initiate the next phase in the project.

5.5 Managing Analysis

In this section, we discuss issues related to managing the analysis activities in a multiteam development project. The primary challenge in managing the requirements in such a project is to maintain consistency while using so many resources. In the end, the requirements analysis document should describe a single coherent system understandable to a single person.

We first describe a document template that can be used to document the results of analysis (Section 5.5.1). Next, we describe the role assignment to analysis (Section 5.5.2). We then address communication issues during analysis. Next, we address management issues related to the iterative and incremental nature of requirements (Section 5.5.4).

5.5.1 Documenting Analysis

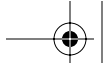
As we saw in the previous chapter, the requirements elicitation and analysis activities are documented in the Requirements Analysis Document (RAD). Sections 1–3.5.2 have already been written during requirements elicitation. During analysis, we revise these sections as ambiguities and new functionality is discovered. The main effort, however, focuses on writing the sections documenting the analysis object model (3.5.3–3.5.4).

Section 3.5.3, Object models, documents in detail all the objects we identified, their attributes, and, in case we used sequence diagrams, operations. As each object is described with textual definitions, relationships among objects are illustrated with class diagrams.

Section 3.5.4, Dynamic models, documents the behavior of the object model, in terms of statechart diagrams and sequence diagrams. Although this information is redundant with the use case model, dynamic models enable us to more precisely represent complex behaviors, including use cases involving many actors.

The RAD, once completed and published, will be baselined and put under configuration management. The revision history section of the RAD will provide a history of changes as a list of the author responsible for the change, date of change, and brief description of the change.



**Requirements Analysis Document**

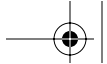
1. Introduction
2. Current system
3. Proposed system
 - 3.1 Overview
 - 3.2 Functional requirements
 - 3.3 Nonfunctional requirements
 - 3.4 Pseudo requirements
 - 3.5 System models
 - 3.5.1 Scenarios
 - 3.5.2 Use case model
 - 3.5.3 Object model
 - 3.5.3.1 Data dictionary
 - 3.5.3.2 Class diagrams
 - 3.5.4 Dynamic models
 - 3.5.5 User interface—navigational paths and screen mock-ups
4. Glossary

5.5.2 Assigning Responsibilities

Analysis requires the participation of a wide range of individuals. The target user provides application domain knowledge. The client funds the project and coordinates the user side of the effort. The analyst elicits application domain knowledge and formalizes it. Developers provide feedback on feasibility and cost. The project manager coordinates the effort on the development side. For large systems, many users, analysts, and developers may be involved, introducing additional challenges during for integration and communication requirements of the project. These challenges can be met by assigning well-defined roles and scopes to individuals. There are three main types of roles: generation of information, integration, and review.

- The **user** is the application domain expert, who generates information about the current system, the environment of the future system, and the tasks it should support. Each user corresponds to one or more actors and helps identify their associated use cases.
- The **client**, an integration role, defines the scope of the system based on user requirements. Different users may have different views of the system, either because they will benefit from different parts of the system (e.g., a dispatcher vs. a field officer) or because the users have different opinions or expectations about the future system. The client serves as an integrator of application domain information and resolves inconsistencies in user expectations.
- The **analyst** is the development domain expert, who models the current system and generates information about the future system. Each analysis is initially responsible for detailing one or more use cases. For a set of use cases, the analysis will identify a number





of objects, their associations, and their attributes, using the techniques outlined in Section 5.4.

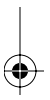
- The **architect**, an integration role, unifies the use case and object models from a system point of view. Different analysts may have different styles of modeling and different views of the parts of the systems that they are not responsible for. Although analysts work together and will most likely resolve differences as they progress through analysis, the role of the architect is necessary to provide a system philosophy and to identify omissions in the requirements.
- The **document editor** is responsible for the low-level integration of the document. The document editor is responsible for the overall format of the document and its index.
- The **configuration manager** is responsible for maintaining a revision history of the document as well as traceability information relating the RAD with other documents (such as the System Design Document; see Chapter 6, *System Design*).
- The **reviewer** validates the RAD for correctness, completeness, consistency, realism, verifiability, and traceability. Users, clients, developers, or other individuals may become reviewers during requirements validation. Individuals that have not yet been involved in the development represent excellent reviewers, because they are more able to identify ambiguities and areas that need clarification.

The size of the system determines the number of different users and analysts that are needed to elicit and model the requirements. In all cases, there should be one integrating role on the client side and one on the development side. In the end, the requirements, however large the system, should be understandable by a single individual knowledgeable in the application domain.

5.5.3 Communicating about Analysis

The task of communicating information is most challenging during requirements elicitation and analysis. Contributing factors include:

- *Different background of participants.* Users, clients, and developers have different domains of expertise and use different vocabularies to describe the same concepts.
- *Different expectations of stakeholders.* Users, clients, and managements have different objectives when defining the system. Users want a system that supports their current work processes, with no interference or threat to their current position (e.g., an improved system often translates into the elimination of current positions). The client wants to maximize return on investment. Management wants to deliver the system on time. Different expectations and different stakes in the project can lead to a reluctance to share information and to report problems in a timely manner.





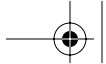
- *New teams.* Requirements elicitation and analysis often marks the beginning of a new project. This translates into new participants and new team assignments, and, thus, into a ramp-up period during which team members must learn to work together.
- *Evolving system.* When a new system is developed from scratch, terms and concepts related to the new system are in flux during most of the analysis and the system design. A term today may have a different meaning tomorrow.

No requirements method or communication mechanism can address problems related to internal politics and information hiding. Conflicting objectives and competition will always be part of large development projects. A few simple guidelines, however, can help in managing the complexity of conflicting views of the system:

- *Define clear territories.* Defining roles as described in Section 5.5.2 is part of this activity. This also includes the definition of private and public discussion forums. For example, each team may have a discussion database as described in Chapter 3, *Project Communication*, and discussion with the client is done on a separate client database. The client should not have access to the internal database. Similarly, developers should not interfere with client/user internal politics.
- *Define clear objectives and success criteria.* The codefinition of clear, measurable, and verifiable objectives and success criteria by both the client and the developers facilitates the resolution of conflicts. Note that defining a clear and verifiable objective is a nontrivial task, given that it is easier to leave objectives open ended. The objectives and the success criteria of the project should be documented in Section 1.3 of the RAD.
- *Brainstorm.* Putting all the stakeholders in the same room and having them quickly generate solutions and definitions can remove many barriers in the communication. Conducting reviews as a reciprocal activity (i.e., reviewing deliverables from both the client and the developers during the same session) has a similar effect.

Brainstorming, and more generally the cooperative development of requirements, can lead to the definition of shared, ad hoc notations for supporting the communication. Storyboards, user interface sketches, and high-level dataflow diagrams often appear spontaneously. As the information about the application domain and the new system accrue, it is critical that a precise and structured notation be used. In UML, developers employ use cases and scenarios for communicating with the client and the users, and use object diagrams, sequence diagrams, and statecharts to communicate with other developers (see Sections 4.4 and 5.4). Moreover, the latest release of the requirements should be available to all participants. Maintaining a live online version of the requirements analysis document with an up-to-date change history facilitates the timely propagation of changes across the project.





5.5.4 Iterating Over the Analysis Model

Analysis occurs iteratively and incrementally, often in parallel with other development activities such as system design and implementation. Note, however, that the unrestricted modification and extension of the analysis model can only result in chaos, especially when a large number of participants are involved. Iterations and increments need to be carefully managed and requests for changes tracked once the requirements are baselined. The requirements activity can be viewed as several steps (brainstorming, solidification, maturity) converging toward a stable model.

Brainstorming

Before any other development activity is initiated, requirements is a brainstorming process. Everything—concepts and the terms used to refer to them—changes. The objective of a brainstorming process is to generate as many ideas as possible without necessarily organizing them. During this stage, iterations are rapid and far reaching.

Solidification

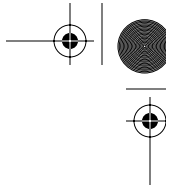
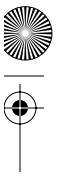
Once the client and the developers converge on a common idea, define the boundaries of the system, and agree on a set of standard terms, solidification starts. Functionality is organized into groups of use cases with their corresponding interfaces. Groups of functionality are allocated to different teams that are responsible for detailing their corresponding use cases. During this stage, iterations are rapid but localized.

Maturity

Changes at the higher level are still possible but are more difficult, and, thus, are made more carefully. Each team is responsible for the use cases and object models related to the functionality they have been assigned. A cross-functional team, the architecture team, made of representatives of each team, is responsible for ensuring the integration of the requirements (e.g., naming).

Once the client signs off the requirements, modification to the analysis model should address omissions and errors. Developers, in particular the architecture team, need to ensure that the consistency of the model is not compromised. The requirements model is under configuration management and changes should be propagated to existing design models. Iterations are slow and often localized.

The number of features and functions of a system will always increase with time. Each change, however, can threaten the integrity of the system. The risk of introducing more problems with late changes results from the loss of information in the project. The dependencies across functions are not all captured; many assumptions may be implicit and forgotten by the time the change is made. Often, the change responds to a problem, in which case there is a lot of pressure to implement it, resulting in only a superficial examination of the consequence of the change. When new features and functions are added to the system, they should be challenged with the following questions: Were they requested by the client? Are they necessary or are they embellishments? Should they be part of a separate, focused utility program instead of part of the



base system? What is the impact of the changes to existing functions in terms of consistency, interface, reliability? Are the changes core requirements or optional features?

When changes are necessary, the client and developer define the scope of the change and its desired outcome and change the analysis model. Given that a complete analysis model exists for the system, specifying new functionality is easier (although implementing it is more difficult).

5.5.5 Client Sign-off

The client sign-off represents the acceptance of the analysis model (as documented by the requirements analysis document) by the client. The client and the developers converge on a single idea and agree about the functions and features that the system will have. In addition, they agree on:

- a list of priorities
- a revision process
- a list of criteria that will be used to accept or reject the system
- a schedule and a budget

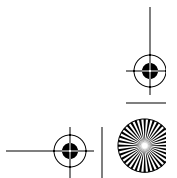
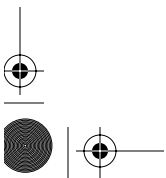
Prioritizing system functions allows the developers to understand better the client's expectations. In its simplest form, it allows developers to separate bells and whistles from essential features of the system. In general, it allows developers to deliver the system in incremental chunks: Essential functions are delivered first, additional chunks are delivered depending on the evaluation of the first chunk. Even if the system is to be delivered as a single, complete package, prioritizing functions enables the client to clearly communicate what is important to her and where the emphasis of the development should be. Figure 5-22 provides an example of a priority scheme.

Each function shall be assigned one of the following priorities

- **High priority**—A high-priority feature must be demonstrated successfully during client acceptance.
 - **Medium priority**—A medium-priority feature must be taken into account in the system design and the object design. It will be implemented and demonstrated in the second iteration of the system development.
 - **Low priority**—A low-priority feature illustrates how the system could be extended in the longer term.
-

Figure 5-22 An example of a priority scheme for requirements.

A revision process enables the client and developer to define how changes in the requirements are to be dealt with after the sign-off. The requirements will change, either because of errors, omissions, changes in the operating environment, changes in the application domain,



or changes in technology. Defining a revision process up front encourages changes to be communicated across the project and reduces the number of surprises in the long term. Note that a change process need not be bureaucratic or require excessive overhead. It can be as simple as naming a person responsible for receiving change requests, approving changes, and tracking their implementation. Figure 5-23 depicts a more complex example in which changes are designed and reviewed by the client before they are implemented in the system. In all cases, acknowledging that requirements cannot be frozen (but only baselined) will benefit the project.

The list of acceptance criteria is revised prior to sign-off. The requirements elicitation and analysis activity clarifies many aspects of the system, including the nonfunctional requirements with which the system should comply and the relative importance of each function. By restating

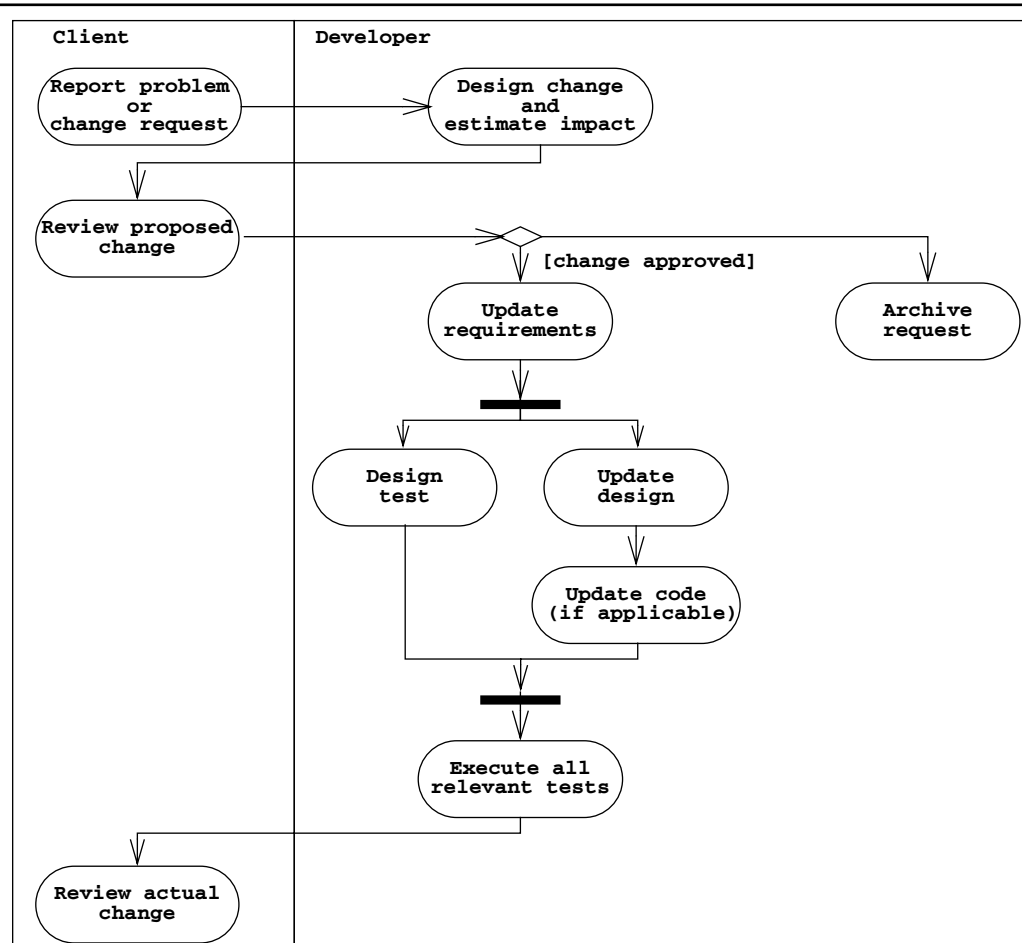


Figure 5-23 An example of a revision process (UML activity diagram).



the acceptance criteria at sign-off, the client ensures that the developers are updated about any changes in client expectations.

The budget and schedule are revisited after the analysis model becomes stable. We describe in Chapter 11, *Project Management*, issues related to cost estimation.

Whether the client sign-off is a contractual agreement or whether the project is already governed by a prior contract, it is an important milestone in the project. It represents the convergence of client and developer on a single set of functional definitions of the system and a single set of expectations. The acceptance of the requirements analysis document is more critical than any other document, given that many activities depend on the analysis model.

5.6 Exercises

1. Consider a file system with a graphical user interface, such as Macintosh's Finder, Microsoft's Windows Explorer, or Linux's KDE. The following objects were identified from a use case describing how to copy a file from a floppy disk to a hard disk: File, Icon, TrashCan, Folder, Disk, Pointer. Specify which are entity objects, which are boundary objects, and which are control objects.
2. Assuming the same file system as before, consider a scenario consisting of selecting a file on a floppy, dragging it to Folder and releasing the mouse. Identify and define at least one control object associated with this scenario.
3. Arrange the objects listed in Exercises 1 and 2 horizontally on a sequence diagram, the boundary objects to the left, then the control object you identified, and finally, the entity objects. Draw the sequence of interactions resulting from dropping the file into a folder. For now, ignore the exceptional cases.
4. Examining the sequence diagram you produced in Exercise 4, identify the associations between these objects.
5. Identify the attributes of each object that are relevant to this scenario (moving a file from a floppy disk to a hard disk). Also consider the exception cases "There is already a file with that name in the folder" and "There is no more space on disk."
6. Consider the object model in Figure 5-24 (adapted from [Jackson, 1995]):
Given your knowledge of the Gregorian calendar, list all the problems with this model. Modify it to correct each of them.
7. Consider the object model of Figure 5-24. Using association multiplicity only, can you modify the model such that a developer unfamiliar with the Gregorian calendar could deduce the number of days in each month? Identify additional classes if necessary.
8. Consider a traffic light system at a four-way crossroads (e.g., two roads intersecting at right angles). Assume the simplest algorithm for cycling through the lights (e.g., all traffic on one road is allowed to go through the crossroad while the other traffic is stopped). Identify the states of this system and draw a statechart describing them. Remember that each individual traffic light has three states (i.e. green, yellow, and red).



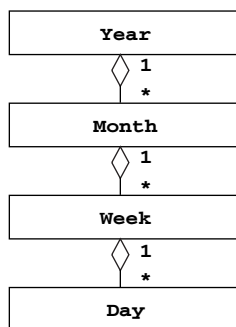
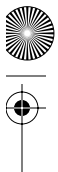
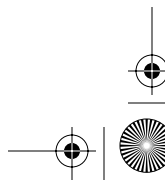
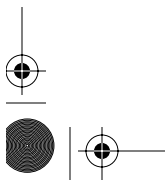


Figure 5-24 A naive model of the Gregorian calendar (UML class diagram).

References

- [Abbott, 1983] R. Abbott, "Program design by informal English descriptions," *Communications of the ACM*, Vol. 26, No. 11, 1983.
- [Booch, 1994] G. Booch, *Object-Oriented Analysis and Design with Applications*, 2nd ed. Benjamin/Cummings, Redwood City, CA, 1994.
- [Booch et al., 1998] G. Booch, J. Rumbaugh, & I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, Reading, MA, 1998.
- [Bruegge et al., 1994] B. Bruegge, K. O'Toole, & D. Rothenberger, "Design considerations for an accident management system," *Proceedings of the Second International Conference on Cooperative Information Systems*, M. Brodie, M. Jarke, M. Papazoglou (eds.), (University of Toronto Canada), pp. 90–100, May 1994.
- [De Marco, 1978] T. De Marco, *Structured Analysis and System Specification*, Yourdon, New York, 1978.
- [Fowler, 1997] M. Fowler, *Analysis Patterns: Reusable Object Models*, Addison-Wesley, Reading, MA, 1997.
- [FRIEND, 1994] *FRIEND Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1994.
- [Harel, 1987] D. Harel, "Statecharts: A Visual Formalism for Complex Systems," *Science of Computer Programming*, pp. 231–274, 1987.
- [Jacobson et al., 1992] I. Jacobson, M. Christerson, P. Jonsson, & G. Overgaard, *Object-Oriented Software Engineering—A Use Case Driven Approach*. Addison-Wesley, Reading, MA, 1992.
- [Jacobson et al., 1999] I. Jacobson, G. Booch, & J. Rumbaugh, *The Unified Software Development Process*. Addison-Wesley, Reading, MA, 1999.
- [Jackson, 1995] M. Jackson, *Software Requirements & Specifications: A Lexicon of Practice, Principles and Prejudices*. Addison-Wesley, Reading, MA, 1995.
- [Larman, 1998] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design*. Prentice Hall, Upper Saddle River, NJ, 1998.
- [Meyer, 1997] Bertrand Meyer, *Object-Oriented Software Construction*, 2nd ed. Prentice Hall, Upper Saddle River, NJ, 1997.
- [Rumbaugh et al., 1991] J. Rumbaugh, M. Blaha, W. Premerlani, F. Eddy, & W. Lorensen, *Object-Oriented Modeling and Design*. Prentice Hall, Englewood Cliffs, NJ, 1991.
- [Wirfs-Brock et al., 1990] R. Wirfs-Brock, B. Wilkerson, & Lauren Wiener, *Designing Object-Oriented Software*. Prentice Hall, Englewood Cliffs, NJ, 1990.





6

6.1	Introduction: A Floor Plan Example	168
6.2	An Overview of System Design	170
6.3	System Design Concepts	172
6.3.1	Subsystems and Classes	173
6.3.2	Services and Subsystem Interfaces	174
6.3.3	Coupling and Coherence	174
6.3.4	Layers and Partitions	178
6.3.5	Software Architecture	182
6.3.6	UML Deployment Diagrams	188
6.4	System Design Activities: From Objects to Subsystems	190
6.4.1	Starting Point: Analysis Model for a Route Planning System	190
6.4.2	Identifying Design Goals	193
6.4.3	Identifying Subsystems	196
6.4.4	Mapping Subsystems to Processors and Components	198
6.4.5	Defining Persistent Data Stores	203
6.4.6	Defining Access Control	207
6.4.7	Designing the Global Control Flow	212
6.4.8	Identifying Boundary Conditions	216
6.4.9	Anticipating Change	218
6.4.10	Reviewing System Design	220
6.5	Managing System Design	221
6.5.1	Documenting System Design	222
6.5.2	Assigning Responsibilities	224
6.5.3	Communicating about System Design	224
6.5.4	Iterating over the system design	226
6.6	Exercises	227
	References	229

